ISA 345 Final Project Report

Emmi Anderson, Brynna Fitzgerald, Bella Hughes, Joseph Sher, Sydney Soler

ISA 345 Section B

11/30/25

INTRODUCTION/PROJECT OVERVIEW

The Miami University women's hockey team and coaches have been struggling to track players, games, practices, and equipment. Throughout this report, we will outline a sports management tracking system to improve organization and allow for quick access to team information. Within this database, entities trace many factors that contribute to a team's performance on a daily basis, namely: player, team, coach, game, statistic, equipment, equipment assignment, and injury report.

Using this database could help facilitate logistics and easily track how players may be performing. Team performance management is crucial because finding insights from data could help improve overall team output and effectiveness. The multiple factors that affect a team's everyday operations and performance are documented by entities in this database including: player, team, coach, game, player statistic, injury, team, practice, and equipment.

Some of the operational transactions that use the sports management database include recording games and updating player statistics (goals, assists, penalties, shots on goal), logging player injuries, assigning equipment (helmet, jerseys, sticks, skates, gloves, bag), and tracking the schedule.

This report will show how the system facilitates day-to-day team operations, this report will provide further information about the database structure, including the entity-relationship diagram (ERD), relationships between entities, and sample queries. The ultimate goal of this project is to provide a dependable and effective system that supports the Miami University Women's Hockey Team's overall success while also improving performance management and data accessibility.

DATABASE DESIGN & IMPLEMENTATION

Exhibit 1: First Draft of the Database Design

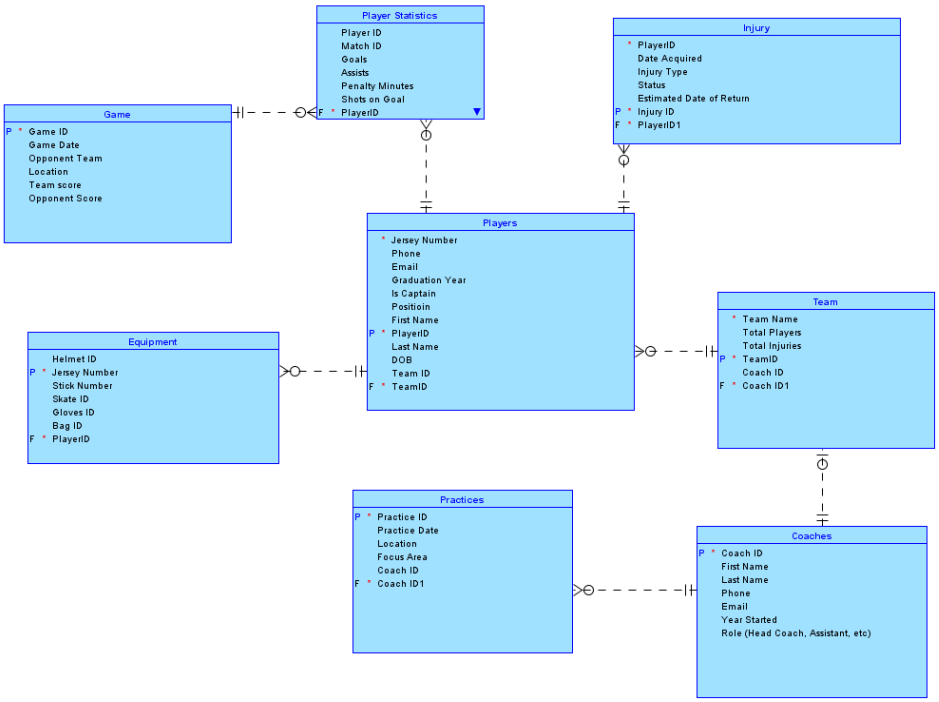


Exhibit 1 represents the logical model of the hockey team's database including eight entities. The database is designed around table Players. Players are the core of teams and it includes a unique identifier of Player ID's along with foreign keys for the team they belong to and their specific jersey number. The reason jersey number is not the unique identifier is because many teams could have the same jersey number.

Players connect to the Team with a many to one relationship as there can be multiple players on a single team while a team can technically have zero players. A player can have a minimum of one team and a maximum of that same team. This table includes team specific information such as name, coach ID to connect to coaches,

Each team can have a minimum of one coach and have more than one coach. A coach can have zero teams, for example, an administrator who oversees everyone but is technically a coach. One coach cannot have more than one team. Within this entity, the unique identifier is Coach ID and then details are outlined such as name, phone, the year they started coaching, and the role they play.

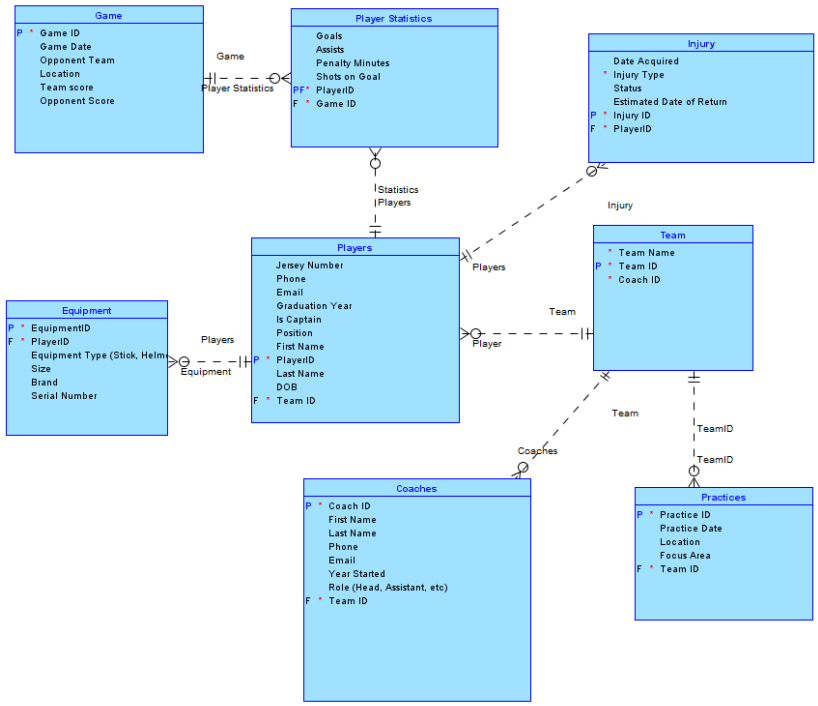
The coach will host either no practices or multiple practices but there will only be one coach assigned per practice. The practice table contains information regarding the date, location, and the focus area. The focus area may be things such as defense, offense, goals, etc.

Moving back to players, the Equipment table is joined to players on Player ID. This is because one player is assigned helmets, jerseys, sticks, gloves, etc. A single player can have no equipment if they are new to the system and haven't received anything yet, or they can have multiple items such as skates.

The injury table is also brought to the players based on Player ID. This describes the players that are unable to play and when they may return to the ice. The minimum and maximum players an injury can be associated with is one. But, a player may have zero injuries or multiple. This relationship is considered optional.

Another optional relationship is Players to Player Statistics. This is optional as an injured player is not going to have specific game statistics. A player can have multiple statistics for multiple games but one set of statistics is going to be associated with one specific player. Building off of player statistics is the Game table which holds details about the game score, opponent, and location. One game can have many statistics (or none), but a statistic will belong to the minimum and maximum of one game.

Exhibit 2: Normalized Model



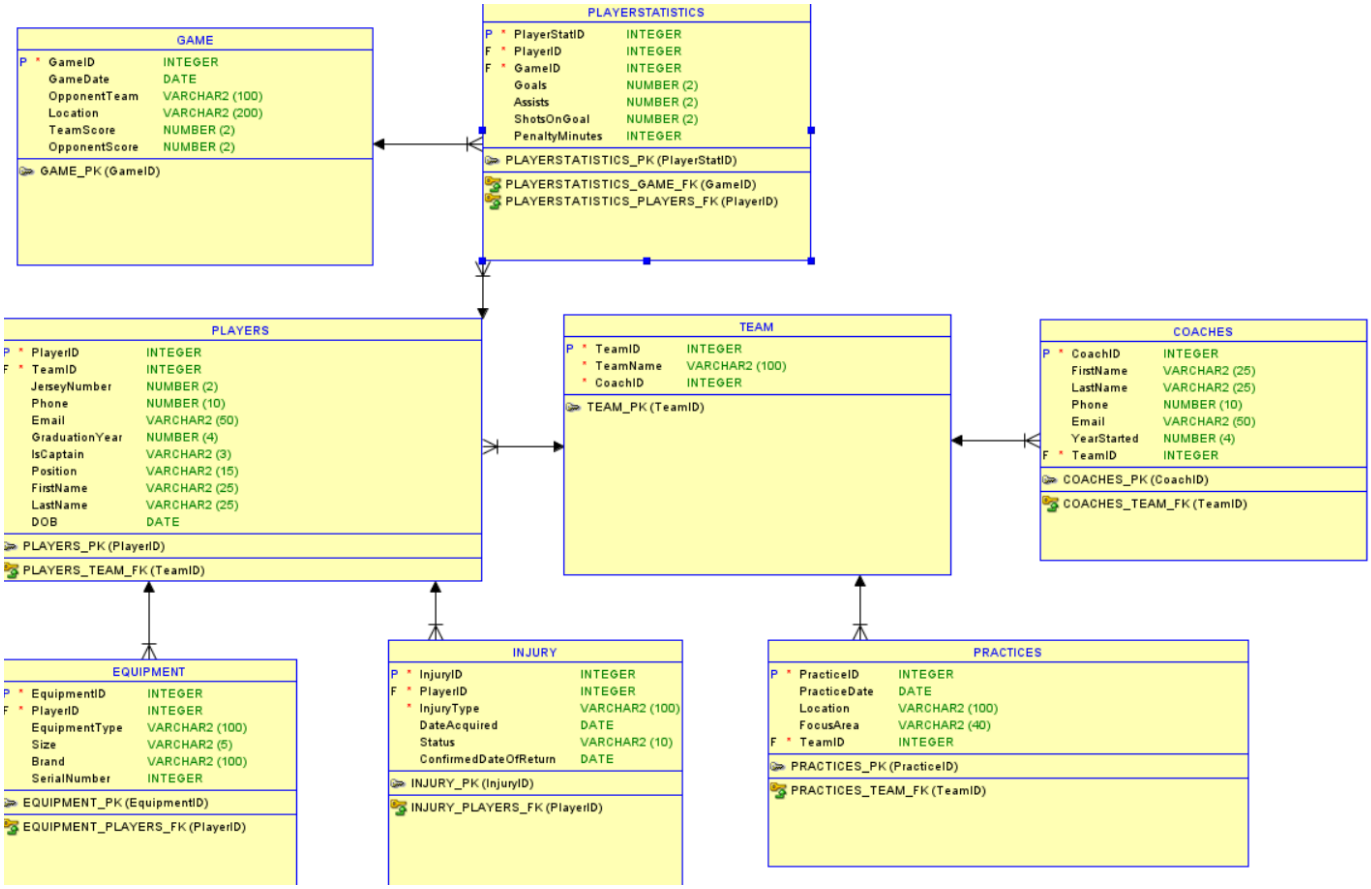
We normalized our database model to Third Normal Form (3NF). This includes making it free of redundancy, protecting it from anomalies, and ensuring that data was structured clearly. First, we had to ensure all eight tables were satisfied by First Normal Form (1NF) so that all attributes are not repeating. In Exhibit 1, some tables say CoachID1 which is not correct. Now, the Players

table, for example, has single-valued attributes such as First Name, Last Name, Position, Jersey Number that are uniquely identified by the Player ID.

The next step we took was evaluating the entities for Second Normal Form (2NF) to ensure the non-key attributes were not partially dependent on part of a composite primary key. The Player Statistics table contains the composite primary key of both Player ID and Game ID. The attributes, Goals, Assets, Penalty Minutes, and Shots on Goal all depend on the composite key rather than just one key individually. This ensures that 2NF has been satisfied by the model.

To achieve Third Normal Form (3NF), the derived fields were removed from the tables. Exhibit 1 included fields such as Total Injuries and Total Players, however, these fields can be derived by querying the data rather than stated directly in the table. Each attribute now depends solely on the primary key of its table, and no fields can be derived from others within a table. Another thing that caught our attention during this process was the relationship between Coaches and Practices. We decided to move it to Teams and Practices as a coach can do a variety of practices for a variety of teams. Moving this simplified the model and makes more logical sense.

Exhibit 3: Physical Model



OPERATIONAL SQL

Subquery and INNER JOIN: This query uses a subquery and inner join to show which players have scored more goals than the team average in goals. This output can be helpful to coaches and scouts to determine which players have the biggest impact on the team, which players should get more ice-time, and which players scouts should track more.

```
SELECT JerseyNumber, FirstName, LastName, Goals, Assists
FROM PLAYERS INNER JOIN PLAYERSTATISTICS ON PLAYERS.PlayerID =
PLAYERSTATISTICS.PlayerID
WHERE Goals > (SELECT AVG(Goals) FROM PLAYERSTATISTICS) OR Assists >
(SELECT AVG(Assists) FROM PLAYERSTATISTICS)
ORDER BY JerseyNumber;
```

OUTER JOIN:

This first query uses outer join to show returning players who have an injury. This query is useful to determine which players will be ready for a given game, and which ones have an injury that will prevent them from playing. This returns the players who have had an injury in the past.

```
SELECT FirstName, LastName, InjuryType, Status
FROM PLAYERS LEFT OUTER JOIN INJURY ON PLAYERS.PlayerID = INJURY. PlayerID
WHERE INJURY.PlayerID IS NOT NULL
ORDER BY FirstName, LastName;
```

This next query is helpful in determining if data entry for a game is incomplete. It is checking which games had no recorded statistics, which signals an error and can help determine which game records must be updated.

```
SELECT GAME.GameID, GameDate, OpponentName
FROM GAME
LEFT OUTER JOIN PlayerStatistics ON GAME.GameID = PLAYERSTATISTICS.GameID
WHERE PLAYERSTATISTICS.GameID IS NULL;
```

This last query determines which players have never attended a practice, as well as the rest of players assigned to teams. If a player does not attend practice, it should result in less playing time, more disciplinary action, and private training lessons for that player to be caught up.

```
SELECT PlayerID, FirstName, LastName, PracticeID
FROM PLAYERS
LEFT OUTER JOIN TEAM ON TEAM.TeamID = PLAYERS.TeamID
LEFT OUTER JOIN PRACTICES ON TEAM.TeamID = PRACTICES.TeamID
```

```
WHERE PracticeID IS NULL
ORDER BY FirstName, LastName;
```

GROUP BY without HAVING: This is helpful because it shows which coaches hold the most practices and are most important in helping the team in practices.

```
SELECT CoachID, FirstName, LastName, COUNT(PracticeID)
FROM COACHES INNER JOIN TEAM ON COACHES.CoachID = TEAM.CoachID
INNER JOIN PRACTICES ON TEAM.TeamID = PRACTICES.TeamID
GROUP BY CoachID, FirstName, LastName
ORDER BY COUNT(PracticeID) DESC;
```

NORMAL: This query is helpful for determining which games, and against which opponents, the hockey team has won. This can be further developed by determining the amount of times the team has won.

```
SELECT GameID, GameDate, TeamScore, OpponentScore
FROM GAME
WHERE TeamScore > Opponent Score;
```

GROUP BY with HAVING: This query is helpful because it is important to track how many pieces of equipment each player has so that they bring the right amount, and don't lose equipment; equipment tracking logistics

```
SELECT FirstName, LastName, COUNT(EquipmentID) AS NumberOfEquipment
FROM PLAYERS INNER JOIN EQUIPMENT ON PLAYERS.PlayerID =
EQUIPMENT.PlayerID
GROUP BY FirstName, LastName
HAVING COUNT(EquipmentID) > 1;
```

SUBQUERY: This is showing players who have more than the average team penalty minutes. Determining which players have more penalty minutes than average can help take better corrective actions to reduce penalties.

```
SELECT JerseyNumber, FirstName, LastName, PenaltyMinutes
FROM PLAYERS INNER JOIN PLAYERSTATISTICS ON PLAYERS.PlayerID =
PLAYERSTATISTICS.PlayerID
WHERE PenaltyMinutes > (SELECT AVG(PenaltyMinutes) FROM PLAYERSTATISTICS);
```

INDEXES: The indexes created are important because they speed up data retrieval and make the overall operational function of the database much quicker in performance. Here, the indexes we created are:

CREATE INDEX PlayerIDIndex ON PLAYERSTATISTICS(PlayerID);

- This index was created because multiple queries join the entities PLAYERS and PLAYERSTATISTICS. In order to speed up the generation of this data, this INDEX helps improve query performance.

CREATE INDEX PlayerInjuryIndex ON INJURY(PlayerID);

- Since the query has an OUTER JOIN, the index is helpful in improving medical status lookups, enhancing reporting of when players can return to play, and facilitating availability decisions.

CREATE INDEX TeamPracticeIndex ON PRACTICES(TeamID);

- The query for the GROUP BY w/o HAVING uses a COUNT, this index helps speed up: loading practice reports, evaluating coaches, and team scheduling

VIEW:

```
CREATE VIEW NONTEAMPERSONNELINJURYCOUNT AS
    SELECT PLAYERS.PlayerID, FirstName, LastName, COUNT(InjuryID)
    FROM PLAYERS
    INNER JOIN INJURY ON PLAYERS.PlayerID = INJURY.PlayerID
    GROUP BY PLAYERS.PlayerID, FirstName, LastName;
```

Teams typically want to keep injury information, such as type of injury and date of return, as disclosed from the public as possible. This view was created because it allows for a virtual table that merely only counts the number of injuries a certain player has instead of disclosing the actual injury information, which may be sensitive. Teams want to keep medical information private, so this view would be created for non-team personnel and the public. Keeping information about injury type, medical treatment, and time of return hidden is important because it helps protect player medical records and history, limiting this private, medical information for just coaches and team-affiliated medical personnel.

```
CREATE VIEW PUBLICPLAYERVIEW AS
    SELECT FirstName, LastName, JerseyNumber, Position, TeamID
    FROM PLAYERS;
```

This next VIEW statement is meant to hide personal information related to players such as their date of birth, their email, phone number, and graduation year. This information may be sensitive

to the players, and it is important to keep this information disclosed within the team only. The public knows information about their first name, last name, jersey number, position, and the team they belong to, but there should be no further information such as phone and email address shown to them. This protects the players' privacy and prevents anyone other than team management from contacting the players and getting a hold of confidential information.

TRIGGER:

```
CREATE TRIGGER PlayerInjuryUpdate
        AFTER
        UPDATE OF Status ON INJURY
        FOR EACH ROW
        WHEN (:NEW.Status = 'Out')
        BEGIN
                UPDATE INJURY
                SET ConfirmedDateOfReturn = :NEW.DateAcquired + 21
                WHERE InjuryID = NEW.InjuryID;
        END;
```

Here, this TRIGGER statement updates rows automatically if a player's status is 'Out'. The confirmed date of return is then pushed back 21 days after the date the player acquired the injury. This can easily help track when players will be good to go and if they need more time to recover.

To properly use the TRIGGER, the following command helps ensure that when the player's injury status changes to 'OUT', 21 days are added to the date acquired to allow for my time to rest and recover. Given this, to run the trigger:

```
UPDATE INJURY
SET Status = 'Out'
WHERE PlayerID = 14616;
```

Then, to see if the TRIGGER worked, the following query could be run to see whether the player was set back 21 days:

```
SELECT InjuryID, Status, DateAcquired, ConfirmedDateOfReturn, DateAcquired
FROM INJURY
WHERE PlayerID = 14616;
```

PROCEDURE:

```
CREATE PROCEDURE AddNewPlayerAndStats (
        p_PlayerID INTEGER,
        p_TeamID INTEGER,
        p_FirstName VARCHAR2(25),
        p_LastName VARCHAR2(25),
        p_JerseyNumber NUMBER(2),
        p_Position VARCHAR2(15)
)
AS
BEGIN
        INSERT INTO PLAYERS(PlayerID, TeamID, FirstName, LastName, JerseyNumber,
        Position)
        VALUES (SELECT MAX(PlayerID) FROM PLAYERS), p_TeamID, p_FirstName,
        p_LastName, p_JerseyNumber, p_Position);

        INSERT INTO PLAYERSTATISTICS(PlayerStatID, PlayerID, Goals, Assists,
        ShotsOnGoal)
        VALUES(SELECT MAX(PlayerStatID) FROM PLAYERSTATISTICS) +1 , p_PlayerID,
        0, 0, 0);
END;
```

This procedure is helpful when a new player joins a team, as when that player joins the team, it automatically creates a blank row for which to enter their statistics. This will run each time a new player is added, so the statistics row can easily be updated. This makes sure that every new player immediately has a statistics record ready to be updated as soon as games are played. This eliminates the need to manually enter new statistics rows as well. To use it in the database, an example query for the procedure could look like:

```
CALL AddNewPlayerAndStats(
        17540,
        101,
        'Sydney',
        'Greenbush',
        45,
        'Center'
);
```

CONCLUSION

Overall, implementation of this database will be helpful for the Women's Hockey Team as it demonstrates the benefit that comes from well-structured, organized, and relational data. This database covers several different aspects of the team that each tie into each other, giving a holistic framework for analysis and insightful information. Specifically, the database provides an advantage since it analyzes multiple aspects such as player performance, equipment distribution, and coaching activities, amongst other things. By organizing this data into a database, the team has a strong starting point for making well-informed decisions and planning for both the short-term and long-term.

From this database, queries were designed to facilitate decision-making and enhance holistic team performance. For example, the queries touched base on areas such as performance, injuries, equipment tracking, and coaching abilities. The performance-based queries demonstrate top players in terms of statistics, as well as determining which players may be lacking in their playing development. This information would help coaches allocate more playing time to top-performers, while giving more training time to those who may be underperforming. Next, the equipment-based query helps track logistics, making sure each player's equipment is transported and kept intact. In this way, there is a reduced risk of losing equipment. Lastly, querying coaches who held the most practices helps determine which coaches should be recognized more and which contribute most to team success. This can be beneficial in deciding whether to keep a coach and promoting those who lead to team development. Combined, these queries provide insightful outcomes that enhance operational functionality. Productivity, and team success.

There were several decisions that helped shape this final project and allowed it to come together. During the design process, one important choice was to normalize the tables. This helped to reduce redundancy and ensure accurate and organized reporting across each of the queries. The second decision made involved restructuring the initial table relationships to hold a better reflection of the team's operations. By separating a player's overall performance statistics from practice sessions, we are able to have a more flexible reporting system. Conducting various tests on each of our queries allowed for the correct any data-entry inconsistencies, which helped resolve any lingering issues and ensure data quality in the system.

Throughout development and implementation of the database, many small adjustments were made, which helped contribute and shape the final product. As a team we were able to build a functional database that is logically structured, query-responsive, and tailored to our client, the Miami Women's Hockey Team, informational/managerial needs.

APPENDIX

DDL for Exhibit 1 Logical Model:

```
-- Generated by Oracle SQL Developer Data Modeler 21.4.2.059.0838
--   at:        2025-11-12 15:44:19 EST
--   site:      Oracle Database 11g
--   type:      Oracle Database 11g


-- predefined type, no DDL - MDSYS.SDO_GEOMETRY

-- predefined type, no DDL - XMLTYPE

CREATE TABLE coaches (
   coach_id                          unknown
--  ERROR: Datatype UNKNOWN is not allowed
    NOT NULL,
   first_name                        unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   last_name                         unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   phone                             unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   email                             unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   year_started                      unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
--  ERROR: Column name length exceeds maximum allowed length(30)
   "Role_(Head_Coach,_Assistant,_etc)" unknown
--  ERROR: Datatype UNKNOWN is not allowed

);

ALTER TABLE coaches ADD CONSTRAINT coaches_pk PRIMARY KEY ( coach_id );

CREATE TABLE equipment (
   helmet_id        unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   jersey_number    unknown
--  ERROR: Datatype UNKNOWN is not allowed
    NOT NULL,
```

```
   stick_number     unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   skate_id         unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   gloves_id        unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   bag_id           unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   players_playerid unknown
--  ERROR: Datatype UNKNOWN is not allowed
    NOT NULL
);

ALTER TABLE equipment ADD CONSTRAINT equipment_pk PRIMARY KEY ( jersey_number );

CREATE TABLE game (
   game_id        unknown
--  ERROR: Datatype UNKNOWN is not allowed
    NOT NULL,
   game_date      unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   opponent_team  unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   location       unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   team_score     unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   opponent_score unknown
--  ERROR: Datatype UNKNOWN is not allowed

);

ALTER TABLE game ADD CONSTRAINT game_pk PRIMARY KEY ( game_id );

CREATE TABLE injury (
   playerid               unknown
--  ERROR: Datatype UNKNOWN is not allowed
    NOT NULL,
   date_acquired          unknown
--  ERROR: Datatype UNKNOWN is not allowed
   ,
   injury_type            unknown
```

```
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   status                unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   estimated_date_of_return unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   injury_id             unknown
-- ERROR: Datatype UNKNOWN is not allowed
    NOT NULL,
   players_playerid      unknown
-- ERROR: Datatype UNKNOWN is not allowed
    NOT NULL
);

ALTER TABLE injury ADD CONSTRAINT injury_pk PRIMARY KEY ( injury_id );

CREATE TABLE player_statistics (
   player_id       unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   match_id        unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   goals           unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   assists         unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   penalty_minutes  unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   shots_on_goal    unknown
-- ERROR: Datatype UNKNOWN is not allowed
   ,
   players_playerid unknown
-- ERROR: Datatype UNKNOWN is not allowed
    NOT NULL,
   game_game_id     unknown
-- ERROR: Datatype UNKNOWN is not allowed
    NOT NULL
);

CREATE TABLE players (
   jersey_number   unknown
-- ERROR: Datatype UNKNOWN is not allowed
    NOT NULL,
   phone           unknown
```

```
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  email          unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  graduation_year unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  is_captain     unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  positioin      unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  first_name     unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  playerid       unknown
-- ERROR: Datatype UNKNOWN is not allowed
   NOT NULL,
  last_name      unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  dob            unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  team_id        unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  team_teamid    unknown
-- ERROR: Datatype UNKNOWN is not allowed
   NOT NULL
);

ALTER TABLE players ADD CONSTRAINT players_pk PRIMARY KEY ( playerid );

CREATE TABLE practices (
  practice_id    unknown
-- ERROR: Datatype UNKNOWN is not allowed
   NOT NULL,
  practice_date  unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  location       unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  focus_area     unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  coach_id       unknown
```

```
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  coaches_coach_id unknown
-- ERROR: Datatype UNKNOWN is not allowed
   NOT NULL
);

ALTER TABLE practices ADD CONSTRAINT practices_pk PRIMARY KEY ( practice_id );

CREATE TABLE team (
  team_name       unknown
-- ERROR: Datatype UNKNOWN is not allowed
   NOT NULL,
  total_players   unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  total_injuries  unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  teamid          unknown
-- ERROR: Datatype UNKNOWN is not allowed
   NOT NULL,
  coach_id        unknown
-- ERROR: Datatype UNKNOWN is not allowed
  ,
  coaches_coach_id unknown
-- ERROR: Datatype UNKNOWN is not allowed
   NOT NULL
);

CREATE UNIQUE INDEX team__idx ON
  team (
    coaches_coach_id
  ASC );

ALTER TABLE team ADD CONSTRAINT team_pk PRIMARY KEY ( teamid );

ALTER TABLE equipment
  ADD CONSTRAINT equipment_players_fk FOREIGN KEY ( players_playerid )
    REFERENCES players ( playerid );

ALTER TABLE injury
  ADD CONSTRAINT injury_players_fk FOREIGN KEY ( players_playerid )
    REFERENCES players ( playerid );

ALTER TABLE player_statistics
  ADD CONSTRAINT player_statistics_game_fk FOREIGN KEY ( game_game_id )
    REFERENCES game ( game_id );

ALTER TABLE player_statistics
```

```
   ADD CONSTRAINT player_statistics_players_fk FOREIGN KEY ( players_playerid )
      REFERENCES players ( playerid );

ALTER TABLE players
   ADD CONSTRAINT players_team_fk FOREIGN KEY ( team_teamid )
      REFERENCES team ( teamid );

ALTER TABLE practices
   ADD CONSTRAINT practices_coaches_fk FOREIGN KEY ( coaches_coach_id )
      REFERENCES coaches ( coach_id );

ALTER TABLE team
   ADD CONSTRAINT team_coaches_fk FOREIGN KEY ( coaches_coach_id )
      REFERENCES coaches ( coach_id );



-- Oracle SQL Developer Data Modeler Summary Report:
--
-- CREATE TABLE                         8
-- CREATE INDEX                         1
-- ALTER TABLE                         14
-- CREATE VIEW                          0
-- ALTER VIEW                           0
-- CREATE PACKAGE                       0
-- CREATE PACKAGE BODY                  0
-- CREATE PROCEDURE                     0
-- CREATE FUNCTION                      0
-- CREATE TRIGGER                       0
-- ALTER TRIGGER                        0
-- CREATE COLLECTION TYPE               0
-- CREATE STRUCTURED TYPE               0
-- CREATE STRUCTURED TYPE BODY          0
-- CREATE CLUSTER                       0
-- CREATE CONTEXT                       0
-- CREATE DATABASE                      0
-- CREATE DIMENSION                     0
-- CREATE DIRECTORY                     0
-- CREATE DISK GROUP                    0
-- CREATE ROLE                          0
-- CREATE ROLLBACK SEGMENT              0
-- CREATE SEQUENCE                      0
-- CREATE MATERIALIZED VIEW             0
-- CREATE MATERIALIZED VIEW LOG         0
-- CREATE SYNONYM                       0
-- CREATE TABLESPACE                    0
-- CREATE USER                          0
--
-- DROP TABLESPACE                      0
-- DROP DATABASE                        0
```

--
-- REDACTION POLICY 0
--
-- ORDS DROP SCHEMA 0
-- ORDS ENABLE SCHEMA 0
-- ORDS ENABLE OBJECT 0
--
-- ERRORS 60
-- WARNINGS 0

DDL for Exhibit 3: Physical Model

```
-- Generated by Oracle SQL Developer Data Modeler 24.3.1.347.1153
--   at:        2025-11-19 15:03:33 EST
--   site:      Oracle Database 11g
--   type:      Oracle Database 11g



-- predefined type, no DDL - MDSYS.SDO_GEOMETRY

-- predefined type, no DDL - XMLTYPE

CREATE TABLE coaches (
    coachid     INTEGER NOT NULL,
    firstname   VARCHAR2(25),
    lastname    VARCHAR2(25),
    phone       NUMBER(10),
    email       VARCHAR2(50),
    yearstarted NUMBER(4),
    teamid      INTEGER NOT NULL
);

ALTER TABLE coaches ADD CONSTRAINT coaches_pk PRIMARY KEY ( coachid );

CREATE TABLE equipment (
    equipmentid   INTEGER NOT NULL,
    playerid      INTEGER NOT NULL,
    equipmenttype VARCHAR2(100),
    "Size"        VARCHAR2(5),
    brand         VARCHAR2(100),
    serialnumber  INTEGER
);

ALTER TABLE equipment ADD CONSTRAINT equipment_pk PRIMARY KEY ( equipmentid );

CREATE TABLE game (
    gameid        INTEGER NOT NULL,
```

```
    gamedate      DATE,
    opponentteam  VARCHAR2(100),
    location      VARCHAR2(200),
    teamscore     NUMBER(2),
    opponentscore NUMBER(2)
);

ALTER TABLE game ADD CONSTRAINT game_pk PRIMARY KEY ( gameid );

CREATE TABLE injury (
    injuryid             INTEGER NOT NULL,
    playerid             INTEGER NOT NULL,
    injurytype           VARCHAR2(100) NOT NULL,
    dateacquired         DATE,
    status               VARCHAR2(10),
    confirmeddateofreturn DATE
);

ALTER TABLE injury ADD CONSTRAINT injury_pk PRIMARY KEY ( injuryid );

CREATE TABLE players (
    playerid       INTEGER NOT NULL,
    teamid         INTEGER NOT NULL,
    jerseynumber   NUMBER(2),
    phone          NUMBER(10),
    email          VARCHAR2(50),
    graduationyear NUMBER(4),
    iscaptain      VARCHAR2(3),
    position       VARCHAR2(15),
    firstname      VARCHAR2(25),
    lastname       VARCHAR2(25),
    dob            DATE
);

ALTER TABLE players
    ADD CONSTRAINT players_ck_1 CHECK ( iscaptain = 'Yes'
                                OR iscaptain = 'No' );

ALTER TABLE players ADD CONSTRAINT players_pk PRIMARY KEY ( playerid );

CREATE TABLE playerstatistics (
    playerstatid   INTEGER NOT NULL,
    playerid       INTEGER NOT NULL,
    gameid         INTEGER NOT NULL,
    goals          NUMBER(2),
    assists        NUMBER(2),
    shotsongoal    NUMBER(2),
    penaltyminutes INTEGER
);
```

```
ALTER TABLE playerstatistics ADD CONSTRAINT playerstatistics_pk PRIMARY KEY ( playerstatid );

CREATE TABLE practices (
   practiceid   INTEGER NOT NULL,
   practicedate DATE,
   location     VARCHAR2(100),
   focusarea    VARCHAR2(40),
   teamid       INTEGER NOT NULL
);

ALTER TABLE practices ADD CONSTRAINT practices_pk PRIMARY KEY ( practiceid );

CREATE TABLE team (
   teamid   INTEGER NOT NULL,
   teamname VARCHAR2(100) NOT NULL,
   coachid  INTEGER NOT NULL
);

ALTER TABLE team ADD CONSTRAINT team_pk PRIMARY KEY ( teamid );

ALTER TABLE coaches
   ADD CONSTRAINT coaches_team_fk FOREIGN KEY ( teamid )
      REFERENCES team ( teamid );

ALTER TABLE equipment
   ADD CONSTRAINT equipment_players_fk FOREIGN KEY ( playerid )
      REFERENCES players ( playerid );

ALTER TABLE injury
   ADD CONSTRAINT injury_players_fk FOREIGN KEY ( playerid )
      REFERENCES players ( playerid );

ALTER TABLE players
   ADD CONSTRAINT players_team_fk FOREIGN KEY ( teamid )
      REFERENCES team ( teamid );

ALTER TABLE playerstatistics
   ADD CONSTRAINT playerstatistics_game_fk FOREIGN KEY ( gameid )
      REFERENCES game ( gameid );

ALTER TABLE playerstatistics
   ADD CONSTRAINT playerstatistics_players_fk FOREIGN KEY ( playerid )
      REFERENCES players ( playerid );

ALTER TABLE practices
   ADD CONSTRAINT practices_team_fk FOREIGN KEY ( teamid )
      REFERENCES team ( teamid );
```

-- Oracle SQL Developer Data Modeler Summary Report:
--
-- CREATE TABLE 8
-- CREATE INDEX 0
-- ALTER TABLE 16
-- CREATE VIEW 0
-- ALTER VIEW 0
-- CREATE PACKAGE 0
-- CREATE PACKAGE BODY 0
-- CREATE PROCEDURE 0
-- CREATE FUNCTION 0
-- CREATE TRIGGER 0
-- ALTER TRIGGER 0
-- CREATE COLLECTION TYPE 0
-- CREATE STRUCTURED TYPE 0
-- CREATE STRUCTURED TYPE BODY 0
-- CREATE CLUSTER 0
-- CREATE CONTEXT 0
-- CREATE DATABASE 0
-- CREATE DIMENSION 0
-- CREATE DIRECTORY 0
-- CREATE DISK GROUP 0
-- CREATE ROLE 0
-- CREATE ROLLBACK SEGMENT 0
-- CREATE SEQUENCE 0
-- CREATE MATERIALIZED VIEW 0
-- CREATE MATERIALIZED VIEW LOG 0
-- CREATE SYNONYM 0
-- CREATE TABLESPACE 0
-- CREATE USER 0
--
-- DROP TABLESPACE 0
-- DROP DATABASE 0
--
-- REDACTION POLICY 0
--
-- ORDS DROP SCHEMA 0
-- ORDS ENABLE SCHEMA 0
-- ORDS ENABLE OBJECT 0
--
-- ERRORS 0
-- WARNINGS 0

INSERT STATEMENTS:

INSERT INTO GAME VALUES(001, to_date('09/10/2025', 'mm/dd/yyyy'), ‘University of Wisconsin’, ‘Goggin Ice Center’, 5, 3);
INSERT INTO GAME VALUES(002, to_date('9/20/2025', 'mm/dd/yyyy'), ‘University of Illinois’, ‘Illinois Ice Center’, 1, 2);
INSERT INTO GAME VALUES(003, to_date('9/30/2025', 'mm/dd/yyyy'), ‘Ohio State University’, ‘Goggin Ice Center’, 2, 4);

INSERT INTO GAME VALUES(004, to_date('10/5/2025', 'mm/dd/yyyy'), ‘University of Cincinnati’, ‘Cincinnati Ice Center’, 3, 0);
INSERT INTO GAME VALUES(005, to_date('10/10/2025', 'mm/dd/yyyy'), ‘Ohio University’, ‘Goggin Ice Center’, 2, 1);
INSERT INTO GAME VALUES(006, to_date('10/11/2025', 'mm/dd/yyyy'), ‘Ohio University’, ‘Athens Ice Center’, 3, 2);
INSERT INTO GAME VALUES(007, to_date('10/15/2025', 'mm/dd/yyyy'), ‘University of Indiana’, ‘Goggin Ice Center’, 0, 4);
INSERT INTO GAME VALUES(008, to_date('10/22/2025', 'mm/dd/yyyy', ‘University of Michigan’, ‘Ann Arbor Ice Arena’, 1, 4);
INSERT INTO GAME VALUES(009, to_date('10/30/2025', 'mm/dd/yyyy'), ‘St Cloud State University’, ‘Goggin Ice Center’, 0, 3);
INSERT INTO GAME VALUES(010, to_date('11/10/2025', 'mm/dd/yyyy'), ‘University of Toledo’, ‘Toledo Ice Arena’, 5, 1);

INSERT INTO COACHES VALUES(1, 'Michael', 'Thompson', 5135551234, 'mthompson@team.com', 2018, 101);
INSERT INTO COACHES VALUES(2, 'Sarah', 'Johnson', 5135555678, 'sjohnson@team.com', 2020, 102);
INSERT INTO COACHES VALUES(3, 'David', 'Williams', 6145559988, 'dwilliams@team.com', 2015, 103);
INSERT INTO COACHES VALUES(4, 'Emily', 'Martinez', 3305552211, 'emartinez@team.com', 2021, 104);
INSERT INTO COACHES VALUES(5, 'Robert', 'Anderson', 4405553344, 'randerson@team.com', 2017, 105);
INSERT INTO COACHES VALUES(6, 'Jessica', 'Brown', 9375554455, 'jbrown@team.com', 2019, 106);
INSERT INTO COACHES VALUES(7, 'Christopher', 'Davis', 2165555566, 'cdavis@team.com', 2016, 107);
INSERT INTO COACHES VALUES(8, 'Ashley', 'Wilson', 5135556677, 'awilson@team.com', 2013, 108);
INSERT INTO COACHES VALUES(9, 'Matthew', 'Moore', 6145557788, 'mmoore@team.com', 2022, 109);
INSERT INTO COACHES VALUES(10, 'Olivia', 'Taylor', 3305558899, 'otaylor@team.com', 2014, 110);

INSERT INTO TEAM VALUES(101, ‘University of Wisconsin’, 1);
INSERT INTO TEAM VALUES(102, ‘University of Illinois’, 2);
INSERT INTO TEAM VALUES(103, ‘Ohio State University’, 3);
INSERT INTO TEAM VALUES(104, 'University of Cincinnati', 4);
INSERT INTO TEAM VALUES(105, 'Ohio University', 5);
INSERT INTO TEAM VALUES(106, 'University of Indiana', 6);
INSERT INTO TEAM VALUES(107, 'University of Michigan', 7);
INSERT INTO TEAM VALUES(108, 'St Cloud State University', 8);
INSERT INTO TEAM VALUES(109, 'University of Toledo', 9);
INSERT INTO TEAM VALUES(110, 'Miami University', 10);

INSERT INTO PLAYERS VALUES(14615, 110, 34, 5134765832, ‘ava.rensford@miamioh.edu’, 2026, ‘No’, ‘Forward’, ‘Ava’, ‘Rensford’, ‘2007-10-06’);
INSERT INTO PLAYERS VALUES(14616, 101, 12, 51344429876, ‘nora.ellington@miamioh.edu’, 2026, ‘No’, ‘Defender’, ‘Nora’, ‘Ellington’, ‘2007-11-02):
INSERT INTO PLAYERS VALUES(14617, 102, 22, 5139842213, ‘lily.calder@miamioh.edu’, 2026, ‘Yes’, ‘Forward’, ‘Lily’, ‘Calder’, ‘2007-03-14’);
INSERT INTO PLAYERS VALUES(14618, 102, 18, 8256754301, ‘calie.marchand@miamioh.edu’, 2026, ‘Yes’, ‘Goalie’, ‘Calie’, ‘Marchand’, ‘2007-01-28’);
INSERT INTO PLAYERS VALUES(14619, 103, 03, 7491749573, ‘elena.brookshire@miamioh.edu’, 2026, ‘No’, ‘Defender’, ‘Elena’, ‘Brookshire’, ‘2006-05-19);
INSERT INTO PLAYERS VALUES(14620, 104, 75, 7593726573, ‘dari.flint@miamioh.edu’, 2025, ‘No’, ‘Defender’, ‘Dari’, ‘Flint’, ‘2004-08-22’);

INSERT INTO PLAYERS VALUES(13753, 105, 09, 7593729573, ‘maya.larkson@maimioh.edu’, 2026, ‘Yes’, ‘Forward’, ‘Maya’, ‘Larkson’, ‘2007-02-10’);
INSERT INTO PLAYERS VALUES(14622, 101, 27, 5739275739, ‘olivia.stroud@miamioh.edu’, ‘2025’, ‘No’, ‘Mid’, ‘Olivia’, ‘Stroud’, ‘2007-12-01’);
INSERT INTO PLAYERS VALUES(14623, 107, 45, 8572947583, ‘serena.talwyn@miamioh.edu’, 2025, ‘No’, ‘Defender’, ‘Serena’, ‘Talwyn’, ‘2004-06-07’);
INSERT INTO PLAYERS VALUES(14624, 108, 04, 6349278354, josie.thorne@miamioh.edu’, 2027, ‘No’, ‘Forward’, ‘Josie’, ‘Thorne’, ‘2007-09-30’);

INSERT INTO PLAYERSTATISTICS VALUES(1, 14615, 001, 2, 1, 5, 0);
INSERT INTO PLAYERSTATISTICS VALUES(2, 14616, 002, 0, 2, 3, 2);
INSERT INTO PLAYERSTATISTICS VALUES(3, 14617, 003, 1, 1, 4, 0);
INSERT INTO PLAYERSTATISTICS VALUES(4, 14618, 004, 0, 0, 6, 0);
INSERT INTO PLAYERSTATISTICS VALUES(5, 14619, 005, 0, 1, 2, 0);
INSERT INTO PLAYERSTATISTICS VALUES(6, 14620, 006, 1, 0, 3, 4);
INSERT INTO PLAYERSTATISTICS VALUES(7, 13753, 007, 2, 1, 7, 0);
INSERT INTO PLAYERSTATISTICS VALUES(8, 14622, 008, 0, 1, 3, 0);
INSERT INTO PLAYERSTATISTICS VALUES(9, 14623, 009, 1, 0, 5, 2);
INSERT INTO PLAYERSTATISTICS VALUES(10, 14624, 010, 3, 0, 8, 0);

INSERT INTO EQUIPMENT VALUES(30001, 14616, ‘Skates’, ‘10’, ‘Bauer’, 8801123);
INSERT INTO EQUIPMENT VALUES(30002, 14617, ‘Stick’, ‘M’, ‘CCM’, 7745091);
INSERT INTO EQUIPMENT VALUES(30003, 14618, ‘Jersey’, ‘L’, ‘Fanatics’, 6623419);
INSERT INTO EQUIPMENT VALUES(30004, 14619, ‘Gloves’, ‘S’, ‘Warrior’, 9154302);
INSERT INTO EQUIPMENT VALUES(30005, 14620, ‘Skates’, ‘8’, ‘Bauer’, 8032214);
INSERT INTO EQUIPMENT VALUES(30006, 14621, ‘Puck’, ‘N/A’, ‘Inglasco’, 5509844);
INSERT INTO EQUIPMENT VALUES(30007, 14622, ‘Stick’, ‘L’, ‘CCM’, 5509844);
INSERT INTO EQUIPMENT VALUES(30008, 14623, ‘Jersey’, ‘XL’, ‘Fanatics’, 9913040);
INSERT INTO EQUIPMENT VALUES(30009, 14623, ‘Skates’, ‘12’, ‘Bauer’, 7348851);
INSERT INTO EQUIPMENT VALUES(30010, 14624, ‘Stick’, ‘S’, ‘CCM’, 6654492);

INSERT INTO INJURY VALUES(1, 14615, 'Sprained Ankle', TO_DATE('2025-01-05','YYYY-MM-DD'), 'Out', TO_DATE('2025-01-20','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(2, 14616, 'Concussion', TO_DATE('2025-01-10','YYYY-MM-DD'), 'Out', TO_DATE('2025-02-01','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(3, 14617, 'Shoulder Strain', TO_DATE('2025-01-12','YYYY-MM-DD'), 'Day-to-Day', TO_DATE('2025-01-18','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(4, 14618, 'Knee Contusion', TO_DATE('2025-01-15','YYYY-MM-DD'), 'Out', TO_DATE('2025-01-25','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(5, 14919, 'Back Tightness', TO_DATE('2025-01-18','YYYY-MM-DD'), 'Questionable', TO_DATE('2025-01-21','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(6, 14620, 'Wrist Fracture', TO_DATE('2025-01-20','YYYY-MM-DD'), 'Out', TO_DATE('2025-03-15','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(7, 14621, 'Hamstring Pull', TO_DATE('2025-01-22','YYYY-MM-DD'), 'Day-to-Day', TO_DATE('2025-01-28','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(8, 14622, 'Groin Strain', TO_DATE('2025-01-25','YYYY-MM-DD'), 'Out', TO_DATE('2025-02-10','YYYY-MM-DD'));
INSERT INTO INJURY VALUES(9, 14623, 'Bruised Rib', TO_DATE('2025-01-27','YYYY-MM-DD'), 'Questionable', TO_DATE('2025-01-30','YYYY-MM-DD'));

INSERT INTO INJURY VALUES(10, 14624, 'Torn ACL', TO_DATE('2025-01-30','YYYY-MM-DD'), 'Out', TO_DATE('2025-09-01','YYYY-MM-DD'));

INSERT INTO PRACTICES VALUES(001, to_date('10/01/2025', 'mm/dd/yyyy'), ‘Goggin Ice Center’, ‘Defense’, 101)
INSERT INTO PRACTICES VALUES(002, to_date('10/03/2025', 'mm/dd/yyyy'), ‘Goggin Ice Center’, ‘Offense’, 102)
INSERT INTO PRACTICES VALUES(003, to_date('10/05/2025', 'mm/dd/yyyy'), ‘Goggin Ice Center’, ‘Defense’, 103)
INSERT INTO PRACTICES VALUES(004, to_date('10/07/2025', 'mm/dd/yyyy'), 'Goggin Ice Center', 'Offense', 104);
INSERT INTO PRACTICES VALUES(005, to_date('10/09/2025', 'mm/dd/yyyy'), 'Goggin Ice Center', 'Defense', 105);
INSERT INTO PRACTICES VALUES(006, to_date('10/11/2025', 'mm/dd/yyyy'), 'Goggin Ice Center', 'Offense', 106);
INSERT INTO PRACTICES VALUES(007, to_date('10/13/2025', 'mm/dd/yyyy'), 'Goggin Ice Center', 'Defense', 107);
INSERT INTO PRACTICES VALUES(008, to_date('10/15/2025', 'mm/dd/yyyy'), 'Goggin Ice Center', 'Offense', 108);
INSERT INTO PRACTICES VALUES(009, to_date('10/17/2025', 'mm/dd/yyyy'), 'Goggin Ice Center', 'Defense', 109);
INSERT INTO PRACTICES VALUES(010, to_date('10/19/2025', 'mm/dd/yyyy'), 'Goggin Ice Center', 'Offense', 110);

DDL Exhibit 2:

```
-- Generated by Oracle SQL Developer Data Modeler 21.4.2.059.0838
-- at: 2025-11-17 15:46:25 EST
-- site: Oracle Database 11g
-- type: Oracle Database 11g



-- predefined type, no DDL - MDSYS.SDO_GEOMETRY

-- predefined type, no DDL - XMLTYPE

CREATE TABLE coaches (
coach_id unknown
-- ERROR: Datatype UNKNOWN is not allowed
NOT NULL,
first_name unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
last_name unknown
-- ERROR: Datatype UNKNOWN is not allowed
```

```
,
phone unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
email unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
year_started unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
"Role_(Head,_Assistant,_etc)" unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
team_team_id unknown
-- ERROR: Datatype UNKNOWN is not allowed
NOT NULL
);

ALTER TABLE coaches ADD CONSTRAINT coaches_pk PRIMARY KEY ( coach_id );

CREATE TABLE equipment (
equipmentid unknown
-- ERROR: Datatype UNKNOWN is not allowed
NOT NULL,
players_playerid unknown
-- ERROR: Datatype UNKNOWN is not allowed
NOT NULL,
-- ERROR: Column name length exceeds maximum allowed length(30)
"Equipment_Type_(Stick,_Helmet,_Gloves,_etc.)" unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
"Size" unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
brand unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
serial_number unknown
-- ERROR: Datatype UNKNOWN is not allowed

);

ALTER TABLE equipment ADD CONSTRAINT equipment_pk PRIMARY KEY ( equipmentid );

CREATE TABLE game (
game_id unknown
-- ERROR: Datatype UNKNOWN is not allowed
NOT NULL,
game_date unknown
-- ERROR: Datatype UNKNOWN is not allowed
```

,
opponent_team unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
location unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
team_score unknown
-- ERROR: Datatype UNKNOWN is not allowed
,
opponent_score unknown
-- ERROR: Datatype UNKNOWN is not allowed

);

ALTER TABLE game ADD CONSTRAINT game_pk PRIMARY KEY (game_id);

CREATE TABLE